arXiv:2608.11668v4 [cs.AR] 14 Sep 2026

HBF Sucks? A Full-Stack Characterization of High-Bandwidth Flash for KV-Centric LLM Serving

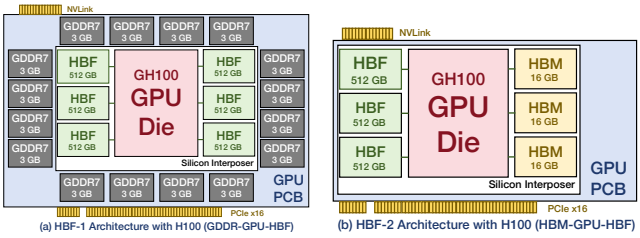
Zhuoran Li[1] Zhuohang Bian[1] Xin Huang[2] Yibo Zhao[1] Guangyu Sun[1] Youwei Zhuo[1,*]
[1]School of Integrated Circuits, Peking University, Beijing, China
[2]Fudan Institute of Systems for Advanced Computing, Fudan University, Shanghai, China
{zhuoranli, bianzhuohang26, yibozhao}@stu.pku.edu.cn {gsun, youwei}@pku.edu.cn huangxin25@m.fudan.edu.cn *Corresponding author
Simulator: https://github.com/pku-lemonade/TokenSim/tree/hbf

A Challenge to the Reader

Every clue is fairly presented below: the traces, models, code, and three necessary conditions. Mooncake-style KV offload fails all three—but HBF need not. This paper already supplies part of the solution. Can you complete the design—an HBF organization and runtime that accelerates critical-path reads, earns enough useful reads per write, sustains bandwidth within thermal and endurance limits, and delivers an end-to-end win?

Abstract

A faster storage device should make serving faster. We find the opposite. High-Bandwidth Flash (HBF) stacks NAND behind a wide, package-local interface, promising flash-scale capacity with far lower read latency and higher bandwidth than an SSD. The obvious move is to keep an SSD-style Mooncake KV-offloading stack and swap in HBF underneath. We built that system and measured it: an extended TokenSim, four complete two-hour Qwen-Bailian production traces, five dense and mixture-of-experts models, and H100/B200 profiles. The upgrade backfires. Average end-to-end latency rises 2–5.5× and maximum SLO goodput falls 1.1–2.7× across H100 and B200, so the faster device yields a slower system. A cost-benefit model explains the paradox: a faster far tier pays off only when read I/O is the bottleneck, reads outweigh writes, and delivered bandwidth is sustainable. Transient KV violates all three at once. Buying flash through the package costs GPU near-tier capacity and bandwidth, while HBF's own read/write latency barely matters: scaling it 3.75× moves latency less than 1%. Worse, the two-tier hierarchy keeps reuse in the near tier and hands HBF a relentless write-heavy stream. Writes outnumber reads on every trace, so a 3D-ICE model shows the stack hits its thermal limit well below peak bandwidth, and a TLC tier wears out sooner than the SSD pool it replaced. The device is fine; the drop-in deployment is not. HBF sucks as an SSD replacement for transient KV, but earns its place in LLM serving when used selectively with reuse-aware placement, write budgeting, and thermal coordination.



(a) HBF-1(2028) (b) HBF-2(2030)

Figure 1: Modeled layouts based on the KAIST HBF roadmap.

Keywords

large language models, KV cache, high-bandwidth flash, memory systems, storage systems, LLM serving

1 Introduction

Large language model (LLM) inference stores a key and value vector for every retained token. The key–value (KV) cache grows with context length, batch size, and active requests, limiting concurrency and retained context. Paged allocation, quantization, and selective retention reduce waste or bytes per token [8, 15, 18, 28] but not the capacity limit. When the working set of prefixes and histories exceeds GPU memory, the system must place KV in a larger tier.

Modern serving systems offload reusable KV to a larger tier. Reuse comes from repeated dialogue, common prefixes in shared prompts and documents, and context revisited by agent workflows. Mooncake and related systems therefore extend the KV pool into

Table 1: HBF topology and evaluation comparison [11].

Work	HBF-resident data	HBF topology	Architecture simulator	Serving evaluation
H^3 [7]	Read-only weights + shared KV	HBF-3	Analytical	Controlled batches
FlashAccel [25]	Weights + KV cache	HBF-2 / HBF-3	Hybrid model	Controlled sweeps
FLINT [19]	Read-only model weights	HBF-3	Trace-coupled	Trace-derived decode
DASH [12]	Expert weights + dynamic KV	HBF-2 + HBF-3	Analytical	Controlled batching
HBFSim [10]	GPU data; expert-weight slice	GPU-visible HBF	Hardware-coupled	Runtime replay
Exploring HBF [22]	Model weights + KV	HBF-2 / HBF-only	Analytical	Controlled batching
This work	**Transient KV; SSD-style offload**	**HBF-1 + HBF-2**	**Full-stack TokenSim + 3D-ICE**	**✓ Production replay: 4 × 2-h Qwen traces**

CPU DRAM, remote memory, and SSDs [5, 20]. This offloading is standard and essential: without it, a capacity-limited server must reject requests or recompute evicted KV. It supplies the objects that larger tiers store and restore.

High-Bandwidth Flash (HBF) connects stacked NAND to the accelerator through a wide memory interface. We base our evaluation on the HBF roadmap's near-term organizations, HBF-1 (2028) and HBF-2 (2030) [11]. HBF-1 pairs package-local HBF with a GDDR7 near tier; HBF-2 shares package sites between HBM and HBF. These dates denote roadmap projections; HBF-3 follows in 2032.

HBF studies demonstrate benefits for read-mostly objects and specialized data paths [7, 9]. We take the interface and management requirements directly from the OCP base-die specification: AXI over UCIe, page-oriented writes, and host-directed data layout [1]. We study HBF under an unmodified Mooncake-style SSD KV-pooling runtime: admission, retention, lookup, and KV save and restore remain unchanged while the backing tier and transport change. Faster flash service thus arrives with changes in near-tier capacity and bandwidth. The pooling policy determines which objects reach HBF; admission and queueing determine the serving outcome. Table 1 places this evaluation alongside recent HBF studies with other organizations and workloads.

We evaluate this roadmap-constrained deployment under the same runtime, comparing complete HBF-1 and HBF-2 organizations against SSD baselines using TokenSim, four complete two-hour production traces, five dense and mixture-of-experts (MoE) models, and H100/B200 profiles. HBF is slower on every request metric: across H100 and B200, average end-to-end latency rises 2–5.5×, and maximum service-level objective (SLO) goodput falls 1.1–2.7×. Faster flash barely helps; changing HBF read/write latency by 3.75× moves average end-to-end latency by less than 1%. Table 3 summarizes the evaluation workloads, model profiles, and system organizations.

The measured behavior arises from the interaction between the SSD-style hierarchy and the roadmap package tradeoff. The hierarchy sends HBF transient and write-heavy KV, whose traffic exercises the physical limits of NAND. The boundary is simple: faster media helps only when secondary-tier service is on the request critical path (C1), later reads repay each flash write (C2), and delivered bandwidth is sustainable (C3). § 5 formalizes these as three necessary conditions, each with a falsifiable requirement tested by two findings. HBF helps only when all three hold at once, so failing any one sinks the substitution.

Six findings form three matched pairs, each falsifying one condition:

C1—Read I/O is the serving bottleneck. Complete HBF-1 and HBF-2 gain flash capacity by reducing GPU near-tier capacity and bandwidth, so the near tier admits fewer requests while the flash service HBF accelerates is a small fraction of the critical path (Finding 1, § 6.1). Base-die attention compute cannot raise that fraction (Finding 2, § 6.2).

C2—Reads outweigh writes. The two-tier hierarchy serves reuse from the near tier and sends HBF a write-heavy cold stream, so writes outnumber reads on every trace (Finding 3, § 7.1). The SSD write-batching optimization that might lower their cost does not transfer (Finding 4, § 7.2).

C3—Read/write Performance sustains. That stream drives the stack to its thermal limit well below peak bandwidth, forcing plane throttling (Finding 5, § 8.1). Its write volume also exhausts HBF's endurance budget sooner than the capacity-matched SSD pool's budget (Finding 6, § 8.2).

HBF sucks as a drop-in SSD KV pool under this unmodified runtime. The result motivates selective, reuse-aware placement; this paper makes four contributions:

(1) A model of when a faster far-memory tier helps LLM serving, with three necessary conditions: read-I/O exposure, reads per write, and sustained bandwidth, evaluated on roadmap-derived HBF-1 and HBF-2 organizations. It separates media speed from changes in local capacity, bandwidth, and queueing.
(2) Measurements showing why an SSD-style hierarchy sends HBF a write-heavy, low-reuse stream, using per-tier byte counters over four production traces.
(3) A 3D-ICE thermal model and a write-endurance budget that turn HBF's power and wear pressures into sourced, clearly bounded estimates.
(4) Requirements for a critical-path-aware, reuse-aware, write-budgeted, and thermally managed HBF KV tier.

2 High-Bandwidth Flash

We first describe the physical HBF device separately from the software path. The HBF roadmap supplies system organizations and projected generation dates [11]; HAVEN's modeled NAND arrays supply device latency, energy, bandwidth, and package power [9]. The OCP base-die specification v0.7.0, dated August 3, 2026, defines the host interface and management responsibilities [1].

2.1 HBF Organization and Stacking

HBF stacks 3D NAND dies behind wide vertical or package-level interconnects. It also includes a base die or distributed peripheral circuits, many independently accessible NAND regions, and a package-level link to the GPU. Together, these components provide much more internal parallelism and interface bandwidth than a conventional SSD. The key distinction is simple: *HBF changes how NAND connects to the processor; it does not change NAND into DRAM.*

The media hierarchy is

$$\text{stack} \rightarrow \text{die} \rightarrow \text{plane} \rightarrow \text{block} \rightarrow \text{page} \rightarrow \text{cell}. \tag{1}$$

Reads sense pages or subarrays. Programs are slower than reads, and erases use larger blocks. NAND lacks arbitrary in-place overwrites, so invalid pages require later reclamation. Aggregate bandwidth comes from concurrency across channels, dies, planes, and subarrays, not DRAM-like latency for each access.

A representative stack integrates sixteen 128–256-layer NAND dies, through-silicon-via (TSV) arrays, and a base die. The base die concentrates mapping and management logic near the package, a natural control-logic location that also concentrates area and heat (Fig. 2) [11].

The OCP specification defines HBF as a non-coherent, memory-centric device using AXI over UCIe rather than NVMe. The host maps global addresses across independent channels; the base die maps channel-local addresses to NAND and handles ECC, bad blocks,

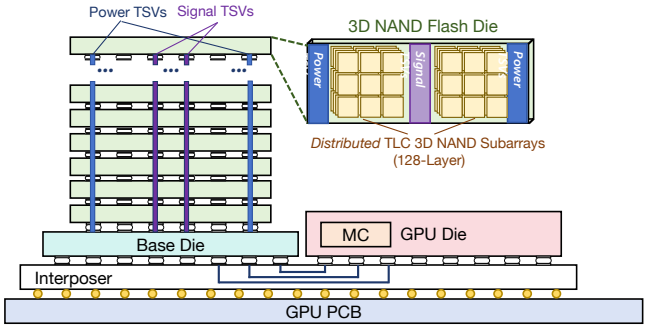


Figure 2: Physical organization of an HBF package.

Table 2: Modeled H100 instances of KAIST's HBF-1/HBF-2 organizations.

Property	HBF-1	HBF-2
Roadmap year	2028	2030
Organization	Single-GPU GDDR–HBF	Local HBM–HBF
Near tier	12 GDDR7, 48 GB	3 HBM, 48 GB
HBF tier	6 stacks, 3 TB	3 stacks, 1.5 TB
Weight tier	GDDR7	HBM
KV movement	On-package GDDR–HBF	Local HBM–HBF

and command scheduling [1, §§3–5]. OCP specifies 64-byte reads, with two page-cache buffers per bank and ordered NAND sensing within each bank. Cache hits can bypass sensing, while batch-read hints control dispatch [1, §5.3].

OCP requires writes to accumulate a complete 4 KiB page and complete only after programming the core die. The host writes pages consecutively within a NAND block; a page-zero write triggers automatic block erase [1, §5.4]. OCP also permits an optional host-managed SRAM scratchpad [1, §11.3]. These mechanisms divide responsibility between the host and base die. Under the SSD-style pooling path, a later reuse still restores KV to the near tier before execution, so the pooling policy controls which reads reach NAND.

2.2 HBF Generations: HBF-1 and HBF-2

Our HBF-1 and HBF-2 profiles instantiate the HBF roadmap's GDDR–HBF and HBM–HBF organizations [11]. Table 2 lists the modeled H100 instances. Table 3(c) also gives the B200 profiles. Stack counts, capacities, and service settings belong to these modeled profiles, while the organizations and generation dates come from the roadmap.

2.2.1 HBF-1. (2028) is a single GPU whose silicon interposer is filled entirely with HBF (Fig. 1a). The interposer can host only a few stacks, so once HBF takes those slots no room is left for HBM, and the fast near tier is pushed off the interposer to GDDR7 in BGA packages on the card PCB. HBF-1 thus buys package-local flash capacity by demoting its near tier from on-interposer HBM to slower off-interposer GDDR.

2.2.2 HBF-2. (2030) shares the interposer between HBM and HBF around one GPU (Fig. 1b). Our modeled equal split assigns half the stack sites to each tier, reducing near-tier capacity and bandwidth relative to an all-HBM package. HBM holds weights and active, write-heavy state while HBF holds long-context KV. One scheduler sees compute, HBM, and HBF over the local memory path, with package area and cooling shared by the two memory tiers.

2.2.3 HBF-3: a later roadmap organization. The roadmap places a later HBF-3 generation in 2032, using an advanced interposer to co-package more HBM and HBF stacks alongside the GPU [11]. Our complete-organization evaluation focuses on the earlier HBF-1 and HBF-2 generations.

Because HBF-1 and HBF-2 change GPU organization and near-tier memory with the medium, a fair study needs two comparisons. The *media-only* comparison varies only secondary-tier latency, bandwidth, and capacity; the *complete-architecture* comparison evaluates the full organization. Without this separation, media service, local memory, interconnect, and package power are inseparable (Section 4).

2.3 Advantages of HBF

2.3.1 Capacity and bandwidth. NAND provides much greater density than DRAM. A 512-GB HBF device paired with a 24-GB HBM stack yields a density ratio of≈ 21.3.

HBF replaces the narrow PCIe/NVMe path with a wide package interface and exposes many NAND regions in parallel. The interface is only one bound on effective application bandwidth:

$$BW_{\text{effective}} = \min(BW_{\text{interface}}, BW_{\text{array}}, BW_{\text{controller}}, BW_{\text{power}}, BW_{\text{thermal}}, BW_{\text{workload}}). \quad (2)$$

Peak bandwidth therefore requires enough independent reads, controller concurrency, and power headroom. The OCP specification makes the layout requirement explicit: the host interleaves data across channels and aligns bank accesses to expose parallelism [1, §§11.1–11.2].

2.3.2 Shorter path and a cooperative hierarchy. HBF removes PCIe/NVMe command processing, host staging, and board links. Modern GDS- and Tutti-style SSD paths narrow this advantage, so we use a strong SSD baseline. In the intended cooperative hierarchy, HBM holds weights, active KV, and write-heavy state; HBF holds larger read-mostly reusable objects; and SSD keeps colder capacity. A NAND read remains slow, but die-, plane-, and channel-level parallelism provides high aggregate throughput when the runtime exposes concurrency.

2.4 Limitations of HBF

The wide interface does not change NAND, so HBF inherits three device-level limits relative to HBM. First, *latency*: NAND reads and writes take on the order of microseconds, roughly two orders of magnitude longer than HBM accesses; programs are slower and more energy-intensive than reads [9]. Die-, plane-, and channel-level parallelism raises aggregate bandwidth but does not close this per-access gap.

Second, *endurance*: flash cells tolerate finite program/erase cycles, and elevated temperature accelerates wear [2]. A write-heavy

tier therefore degrades far faster than DRAM's effectively unbounded write endurance.

Third, *power and thermals*: per-stack HBF power is far higher than HBM. H3 assumes roughly 160 W per HBF cube, so six stacks can approach 960 W [7], potentially exceeding the GPU die's peak power. Package-local HBF therefore creates a new power and thermal problem rather than inheriting HBM's modest envelope.

3 Background

The preceding section introduced the physical HBF device and its roadmap organizations. This section explains how inference creates, reuses, and offloads KV state and why serving systems increasingly move it to a larger tier. It provides the software and storage context for the HBF deployment evaluated here.

3.1 KV Cache and Reuse in Agentic Serving

Agentic and conversational workloads dominate modern LLM deployments: multi-turn dialogue, tool-use loops, shared system prompts, and retrieval-augmented generation (RAG). They revisit tokens, reusing prompt prefixes and session histories across requests. Serving therefore depends less on raw compute than on memory *capacity* to retain useful prefixes and active histories and *bandwidth* to restore or stream them without stalling the GPU.

LLM inference has *prefill* and *decode* phases. Prefill processes prompt tokens in parallel and materializes KV state. A prefix-cache hit avoids recomputing the matched prompt but still requires locating and delivering its saved blocks to the GPU. Decode generates tokens autoregressively and repeatedly reads the retained KV history. Each transformer layer stores a key and value vector per token. For N retained tokens, the unsharded per-request KV footprint is

$$B_{\mathrm{KV/request}} = 2NLH_{\mathrm{KV}}D_{\mathrm{head}}b, \tag{3}$$

Here, L, H_{KV}, D_{head}, and b denote layer count, KV-head count, head dimension, and bytes per element; the factor of two accounts for keys and values. Grouped-query and latent-attention designs reduce this footprint but do not remove the capacity problem.

Unlike static shared weights, KV is generated online, grows during decode, and has request-, session-, and prefix-specific lifetimes. Stored prefixes matter only if later requests reuse them before eviction. Continuous batching adds pressure: admissions consume KV capacity, long requests retain blocks longer, and restoring offloaded state can delay execution. KV placement, reuse, and request scheduling must therefore be optimized together.

3.2 Paged and Pooled KV Storage

3.2.1 PagedAttention. vLLM introduces PagedAttention and block-level KV allocation [15]. A block table maps each request's logical KV blocks to non-contiguous physical blocks in GPU memory. On-demand allocation reduces fragmentation, supports continuous growth, and enables common-prefix sharing and copy-on-write without a large contiguous reservation.

Two requests share exact-prefix KV when their token prefixes and model state match. The benefit grows with matched length but vanishes after eviction. System prompts and multi-turn dialogue create heavy reuse; independent API calls create little. The same admission policy can therefore yield very different read/write behavior across traces.

3.2.2 Mooncake and distributed KV pools. Paged allocation solves on-GPU layout but not capacity beyond HBM. Mooncake extends KV into a distributed, KV-cache-centric pool of CPU DRAM, SSDs, and network resources. It combines a transfer engine with KV-aware, SLO-aware scheduling that rejects work early under overload [20]. *KV-cache-centric scheduling* treats KV-block location, availability, reuse, and transfer cost as first-class state beyond request selection.

SSD offloading is asynchronous and outside the write path. New KV lands in DRAM; after DRAM crosses a watermark, a background policy flushes or evicts it to SSD. A master node tracks per-object locations and promotes hot blocks back [14]. Flash writes are therefore scheduling decisions, separate from GPU compute. Save and load move KV in opposite directions:

$$\text{save:}\quad \text{GPU HBM} \rightarrow \text{cache tier}, \tag{4}$$

$$\text{load:}\quad \text{cache tier} \rightarrow \text{GPU HBM}, \tag{5}$$

Their imbalance is central to the write/read behavior quantified in Section 7.1.

3.3 SSD Offloading of KV

3.3.1 Why SSD offloading helps. HBM and host DRAM are fast but capacity-limited. SSDs add larger, cheaper, replaceable capacity and retain prefixes over the longer reuse windows needed by long-context, multi-turn, RAG, and agent workflows.

DeepSeek's API documents a production disk-based context cache with persisted prefix units and cache-hit/miss token accounting [3]. Production traces show bimodal reuse: most occurs within minutes, but a long tail returns only after tens of minutes. Recency-only eviction therefore discards blocks that would still be reused [14]. SSD offloading is now standard and provides KV objects for larger tiers to store and restore.

3.3.2 Modern offload paths. In a naive SSD path, data moves through host memory (GPU HBM → CPU I/O → NVMe → SSD). Submission overhead, extra copies, and fragmented small requests can then dominate media service (Fig. 3(a)).

Modern paths remove much of this cost (Fig. 3(b)). Linux `io_uring` uses shared queues to reduce system calls and context switches [16]. GPU Direct Storage (GDS) enables direct DMA between NVMe and GPU memory. Tutti moves object identification and completion to a GPU-side `io_uring` path, removing the CPU from each operation's critical path [21].

SSD offloading is therefore a strong, actively optimized baseline. Drives remain replaceable outside the accelerator package, and their controllers provide buffering, mapping, garbage collection, wear leveling, and bad-block management.

SSD remains off-package and bounded by the PCIe/NVMe link. HBF provides a higher-bandwidth, package-local alternative, as described in Section 2. We evaluate HBF by replacing or augmenting the SSD backing tier while preserving the KV-object path above.

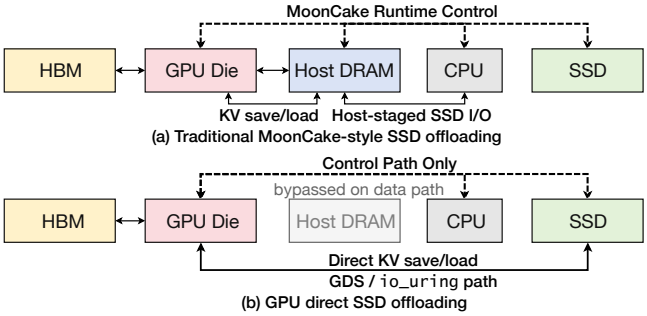


Figure 3: SSD offloading of KV: host-staged path (a) versus GPU-direct path (b).

Table 3: Evaluation workloads, models, and systems.

(a) Production traces

Trace	Requests/QPS	Prompt	Output	Multi-turn/reused
`traceA`	43,058/5.98	1,046/6,436	376/808	46.3%/46.8%
`traceB`	172,800/24.0	574/1,720	39/180	0%/6.0%
`coder`	43,011/5.97	4,540/12,776	469/1,789	38.6%/39.0%
`thinking`	10,812/1.50	3,680/13,115	1,666/9,157	11.1%/14.3%

(b) Model profiles

Model	Arch.	Params	Layers/hidden	Attn./KV heads	Max context
Qwen3-4B	Dense	4B/4B	36/2,560	32/8	40,960
Qwen3-32B	Dense	32B/32B	64/5,120	64/8	40,960
DeepSeek-V3.2-685B	MoE	685B/37B	61/7,168	128/128	163,840
GLM-5.2-753B	MoE	753B/40B	78/6,144	64/64	1,048,576
Kimi-K2.7-Code-1.1T	MoE	1.1T/32B	61/7,168	64/64	262,144

(c) Eight-GPU system profiles

Profile	Local tier per GPU	Shared secondary tier
SSD/H100	96 GB HBM, 3.0 TB/s	12/24/48 TB SSD; 54.7–218.8 GB/s read
SSD/B200	192 GB HBM, 8.0 TB/s	12/24/48 TB SSD; 54.7–218.8 GB/s read
HBF-1/H100	48 GB GDDR7, 1.344 TB/s	24 TB HBF, 1.2 TB/s
HBF-1/B200	96 GB GDDR7, 2.688 TB/s	48 TB HBF, 2.4 TB/s
HBF-2/H100	48 GB HBM3e, 1.5 TB/s	12 TB HBF, 0.6 TB/s
HBF-2/B200	96 GB HBM3e, 4.0 TB/s	24 TB HBF, 1.2 TB/s

4 Evaluation Methodology

4.1 Simulator and Evaluation Setup

We use an extended version of TokenSim[1] [26]. It models request arrival, prefill, decode, continuous batching, paged KV allocation, admission, preemption, recomputation, KV save/restore, tier transfers, queueing, and request-level latency. Our extensions add Mooncake-style lookup, hashed 16-token KV identifiers, object save/load events, SSD and HBF tiers, per-tier read/write counters, HBF-1/HBF-2 profiles, connector wait accounting, and SLO-oriented outputs. TokenSim's original studies validate its serving behavior against real-machine configurations [26]; for our extensions, regression checks cover block/byte conservation, save/load ordering, deterministic replay, and per-tier counters. HBF service parameters are sourced or swept. The complete-architecture campaign instantiates the HBF roadmap's HBF-1 (2028) and HBF-2 (2030), with the capacity and bandwidth settings in Table 3(c).

[1]https://github.com/pku-lemonade/TokenSim

We separate media sensitivity from complete architecture comparisons: the media-only view holds GPU organization and policy fixed while varying HBF latency, while complete HBF-1 and HBF-2 change the near tier to the roadmap's GDDR/HBF and HBM/HBF organizations. HBF latency is swept over 8/80, 12/120, 20/200, and 30/300 μs read/write points. The connector uses 16-token hash blocks: prefix lookup matches consecutive identifiers from the prompt start, missing full blocks are saved during prefill after existing-block filtering, and a later matching request restores them before its missing suffix executes.

4.2 Workloads, Models, and Platforms

We use the four anonymized production-derived traces listed in Table 3 from the Aliyun Qwen-Bailian deployment [24]. Each contains a complete two-hour window with fixed request order, lengths, session structure, and 16-token KV identifiers. `traceA` mixes production traffic with conversational reuse. `traceB` contains short-output API/text traffic without multi-turn sessions, although shared prefixes still create cross-request reuse. `coder` contains code-oriented multi-turn context, and `thinking` combines long reasoning outputs with a smaller multi-turn population. A reused block is a unique hash identifier that appears in more than one request.

The five dense and MoE models listed in Table 3(b) supply the architectural parameters used by the current TokenSim profiles. For MoE models, the parameter count reports both total parameters and parameters activated per token.

We model the eight-GPU systems listed in Table 3(c) with tensor parallelism 8, pipeline and data parallelism 1, and expert parallelism for MoE models. The SSD tier uses 3.2-TB KIOXIA CM7-V drives with 14.0/6.75 GB/s sequential read/write bandwidth and 3 TB modeled usable capacity per drive [13]. The main SSD baseline is the conventional Mooncake connector in the simulator.

We evaluate offered loads from 0.05 to 32 QPS by rescaling request-arrival timestamps while preserving request order, contents, sessions, and KV-reuse relationships.

4.3 Metrics and Statistical Treatment

We report TTFT, TBT, end-to-end latency, delivered request and token throughput, queueing and connector delay, KV hits, blocks and bytes read and written, the write/read ratio, prefix reuse, recomputation, and SLO goodput. For the architecture comparison we average each metric over the trace and model campaign at every system and latency point, because the question is how the complete SSD, HBF-1, and HBF-2 organizations behave, not how one favorable point behaves. The write/read ratio and prefix hit rate in Finding 3 come from the per-tier byte counters over a full two-hour replay of each trace, so they reflect fixed request contents and ordering rather than a sampled window. Zero-read runs are excluded from the ratio because it is undefined, and malformed result files are excluded. Throughout, we label every value as a measured simulator output, a sourced device parameter, or a modeled projection.

4.4 Thermal and Endurance Models

Thermal and endurance models use the serving-trace measurements. For thermal behavior we build a 16-Hi HBF stack similar to HBM4, with 128-layer 3D-NAND TLC dies, and solve its steady-state heat

flow with 3D-ICE [23]. We use read and write energy per 16-token KV block, matching the connector's access granularity, and sweep sustained single-stack bandwidth to obtain dynamic power and peak temperature. We choose 80°C as the model's safe junction temperature and use a throttling policy that closes the hottest, highest-traffic planes and redirects writes to cooler planes once the limit is reached. The OCP specification defines normal operation, light and severe throttling, and shutdown. Its throttling thresholds are configurable [1, §9].

For endurance, we normalize the media-write volume from each two-hour, eight-H100 HBF-2 replay to a daily rate. The capacity-matched SSD reference has four KIOXIA CM7-V 3.2 TB drives [13]; its vendor 3-DWPD rating gives 38.4 TB/day. The HBF tier uses a deliberately favorable TLC endurance budget with unit write amplification, yielding 21.7 TB/day for a five-year target. We compare lifetimes by linear exhaustion of these write budgets under the measured trace traffic. OCP specifies a product-specific maximum P/E count and exposes the average P/E count to the host [1, §9].

5 A Model of When Faster Flash Helps

At matched capacity, an HBF read is faster than an SSD read; that much is uncontested. The question is whether per-access speedup propagates to end-to-end request latency, or is absorbed by the rest of the serving pipeline. This is a system-level question; we formalize it with a single cost–benefit relation. We express the net change an HBF tier makes to end-to-end latency, relative to the SSD tier it replaces, as

$$\Delta_{\mathrm{HBF}} = \underbrace{f\,\Delta t_{\mathrm{media}}}_{\text{critical-path I/O}} - \underbrace{C_{\mathrm{move}}}_{\text{movement/control}} - \underbrace{C_{\mathrm{pkg}}}_{\text{near-tier opportunity}} - \underbrace{C_{\mathrm{var}}}_{\text{throttle/variability}} - \underbrace{C_{\mathrm{life}}}_{\text{endurance/capacity}}. \quad (6)$$

Faster flash improves only the first term, and only in proportion to f, the fraction of end-to-end time spent in *exposed* secondary-tier I/O. Everything else the HBF tier changes (the near tier it displaces, the traffic it must absorb, the heat it generates, the wear it takes) enters as a subtracted cost. The HBF tier is net-positive only when $\Delta_{\mathrm{HBF}} > 0$ under target latency, throughput, power, capacity, and lifetime constraints. We make this concrete by converting each term into a measurable quantity, from which three necessary conditions emerge. Each condition is a *falsifiable prediction*: it states the value a term must take for HBF to help, while a paired finding measures the value it actually takes (Table 4).

C1: Read I/O is the serving bottleneck (f must be large). We decompose end-to-end time as $T = T_{\mathrm{rest}} + t_{\mathrm{io}}$, where t_{io} is the exposed, non-overlapped secondary-tier service and T_{rest} covers compute, near-tier service, batching, and queueing; $f = t_{\mathrm{io}}/T$ is the exposed fraction. To first order, the exposed term scales with per-access latency, so making the medium $k\times$ faster changes end-to-end time by

$$\frac{\Delta T}{T} = f\left(1 - \tfrac{1}{k}\right) \le f. \quad (7)$$

The exposed fraction is a hard ceiling on *any* media improvement; even an infinitely fast tier ($k \to \infty$) cannot beat f. Equation (7) is also invertible, so a media-latency sweep recovers f from a serving measurement directly. §6.1 runs that $k = 3.75\times$ sweep, and §6.2 asks whether moving compute into the device can enlarge f.

Table 4: Each condition is a falsifiable prediction of the model in Eq. (6), tested by two findings for transient KV behind an SSD-style connector in §§6–8.

Cond.	Prediction: HBF helps only if	How we test it
C1	read I/O is the serving bottleneck	media-latency sweep recovers f (F1); base-die NMP tests coverage (F2)
C2	reads outweigh writes ($\rho > \rho^\star$)	per-tier byte counters give delivered ρ (F3); paired batching tests the write cost C (F4)
C3	sustained BW near nominal peak	3D-ICE thermal sweep (F5); write-endurance budget (F6)

C2: Reads outweigh writes ($\rho > \rho^\star$). A NAND program is spent the moment an object enters HBF; that cost is repaid only by later reads that benefit from the faster medium. Let ρ be the useful read bytes returned per byte written at the flash tier, S the exposed time each read byte saves, A the write amplification, and C a per-byte capacity and write-budget cost. Writing is net-positive only when $\rho S > AC$, i.e.

$$\rho > \rho^\star = \frac{AC}{S}. \quad (8)$$

Two facts push $\rho^\star$ above one: the write cost is paid unconditionally, while the repaying reads are speculative, and a single NAND program costs several reads in energy and wear. HBF therefore pays off only as a *read-heavy* tier. For transient KV the hierarchy is expected to deliver the opposite: the near tier retains the high-ρ hot blocks and hands HBF the write-once tail. §7.1 measures the delivered ρ, and §7.2 asks whether the SSD write optimization that would lower C transfers to HBF.

C3: Sustained bandwidth. The bandwidth a workload actually receives is the minimum of interface, array, controller, thermal, and endurance limits (Eq. (2)), which can fall well below the datasheet peak. Of these limits, a write-heavy KV stream is most likely to hit the thermal and endurance bounds. §8.1 measures the bandwidth a stack sustains before it overheats, and §8.2 measures the write volume it takes over its life.

These conditions are device-agnostic. f, ρ, and sustained bandwidth characterize any package-local far tier (CXL-attached flash, other stacked NAND, or near-data flash), not only HBF. The rest of the paper tests them on the most demanding object for HBF, *transient KV* behind an SSD-style save/load connector, pairing two findings with each condition (Table 4).

6 C1: Read I/O Is the Serving Bottleneck

Condition C1 requires the exposed secondary-tier fraction f to be large. Both findings in this section measure f instead of assuming a value for it. Finding 1 recovers f from a media sweep over a

capacity-matched swap, and Finding 2 asks whether moving the compute into the device can raise it.

6.1 Finding 1: HBF Trades GPU Capacity and Bandwidth

An HBF read is roughly an order of magnitude faster than an SSD read at matched capacity, so the swap should make requests faster. Condition C1 says that expectation holds only in proportion to f, which no serving study has measured. Equation (7) is invertible, so a media-latency sweep recovers it while a capacity-matched swap answers what HBF delivers end to end.

Each HBF organization is paired with the SSD baseline of equal secondary-tier capacity, HBF-1 against SSD24 and HBF-2 against SSD12 (Table 3(c)). Both HBF generations give half of the GPU's near tier to flash and hold equal near-tier capacity, differing in its technology (GDDR7 vs HBM3e); HBF-1 also carries twice the flash capacity and twice the flash bandwidth of HBF-2. The second control varies the medium, moving the read/write point from 30/300 to 8/80 μs, the $k = 3.75\times$ change Eq. (7) needs to recover f. Table 5 reports both sweeps for every GPU and retained model.

The sweep shows every HBF configuration worse than its SSD pair on every request metric, with mean end-to-end latency 2–5.5× the baseline, delivered throughput down 4–34%, and maximum SLO goodput down 1.1–2.7×; the H100 pairs sit at the severe end of each range and the B200 pairs at the mild end. Across models and traces every HBF curve rises above SSD12 and saturates at lower offered load (Figs. 5–7).

The media sweep moves almost nothing. A $k = 3.75\times$ improvement in raw media latency shifts mean end-to-end latency by 0.75% (HBF-1) and 0.88% (HBF-2), which inverts through Eq. (7) to $f = (\frac{\Delta T}{T})/(1 - \frac{1}{k}) \approx 1\%$. The media change remains measurable elsewhere, moving throughput ~5% and SLO goodput 14–23%.

Why the faster medium loses. Media service does not reach the request's critical path. On GLM-5.2, `traceA`, B200 at 32 QPS, HBF-1's cumulative connector service time across all requests is 3.2× *lower* than SSD's (399 vs 1,293 s), yet its average per-request end-to-end latency over the replay is 2.6× *higher* (4,386 vs 1,682 s) and its TBT doubles (Fig. 4). Across the campaign 4,741 matched pairs share that shape, so the bottleneck lies outside media service.

Two comparisons locate it, and neither is a single-variable control. The HBF/SSD pair changes near-tier capacity and bandwidth together, halving both (96 to 48 GB and 3.0 to 1.5 TB/s on H100), so it prices the package trade without separating its two parts. The HBF-1/HBF-2 pair instead holds near-tier capacity fixed, and HBF-1 is 16–64% slower end to end despite carrying twice the flash capacity and bandwidth, so a strictly better flash tier does not recover the loss. What the swap degrades is the near tier that governs how many requests the server admits at once.

The result is a ceiling, not a tuning outcome. With $f \approx 1\%$, a perfect zero-latency medium could remove at most one percent of the critical path, while the package trade that buys it costs 2–5.5× in end-to-end latency. In Eq. (6), C_{pkg} exceeds $f\,\Delta t_{\text{media}}$ by two orders of magnitude. One alternative explanation is that f is small only because the compute sits far from the data, which Finding 2 tests directly.

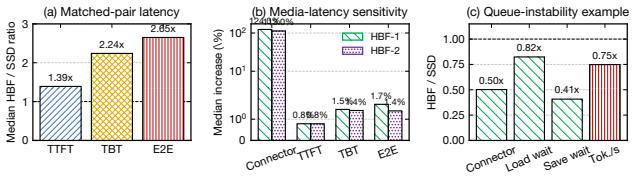


Figure 4: A faster connector, a slower system: HBF cuts transfer time yet raises end-to-end latency because the smaller near tier destabilizes the queue.

Finding 1

Replacing SSD with HBF sacrifices the near-tier capacity and bandwidth that govern admission, so faster flash does not improve end-to-end serving.

6.2 Finding 2: Base-Die Compute Cannot Raise the Exposed Fraction

Finding 1 leaves f near 1%. One reading of that number is that the compute sits too far from the data, in which case moving the compute to the data should raise f. The standard form of that move is a near-memory attention engine on the HBF base die, as HBM-PIM does for DRAM. We ask whether it does.

We model an additional decode-attention engine on the base die under assumptions chosen to favor HBF. It is charged no area, power, or programmability penalty, and every eligible token hits at the device service time, so a negative result is a lower bound. The two modes differ in reach. The HBM engine accelerates all HBM-resident KV; the HBF engine reaches only KV already resident in HBF, and hot HBM-resident KV falls back to GPU attention. Table 6 lists the full configuration.

Fig. 8(a) reports both engines. Averaged over the 40 sweep points, the HBM engine cuts decode-attention time by 42.84%, and it is stable across the whole sweep (42.78–42.86%). The HBF engine averages −0.03% on the same basis, that is, no benefit at all. Individual points swing from −29.05% to +20.49%, and it is *slower* than GPU attention at 16 of 40 points, turning positive only near saturation. The end-to-end effect is smaller again. Replacing the HBF engine with the much stronger HBM engine moves decode p50 by 0.39% and decode p99 by 0.82% on non-saturated points.

Two quantities account for this behavior. The first is *coverage*. A base-die engine sees only the cold KV the runtime has already placed in HBF, and the mean HBF-resident fraction is just 15.48% (Fig. 8(b)), so even a perfect HBF engine cannot touch the common HBM-resident case. Table 7 tracks residency and leverage across traces. `thinking` keeps only 4% of attention in HBF and sees essentially no benefit. `traceA` at 27% residency has the most eligible work, yet it is still net-negative, which brings in the second quantity, *scale*. The HBF path pays a 12 μs access on every step, and many low-load decode steps are too small to amortize it. The end-to-end numbers follow the same accounting, since MoE compute, expert communication, prefill, HBF transfers, and queueing are unchanged by either engine.

Table 5: B200 HBF performance relative to capacity-matched SSD baselines. HBF-1 is normalized to SSD24 and HBF-2 to SSD12. Positive latency changes indicate slowdowns; negative throughput and goodput changes indicate losses.

Model	ΔTTFT		ΔE2E Latency		ΔThroughput		ΔGoodput	
	HBF-1	HBF-2	HBF-1	HBF-2	HBF-1	HBF-2	HBF-1	HBF-2
Kimi-K2.7	+96.6%	+44.7%	+285.2%	+131.4%	−24.3%	−12.7%	−51.7%	−41.7%
GLM-5.2	+94.3%	+45.9%	+280.0%	+133.2%	−24.6%	−13.7%	−37.5%	−12.5%
DeepSeek-V3.2	+96.5%	+44.7%	+284.7%	+131.3%	−24.2%	−12.7%	−51.7%	−41.7%
Qwen3-32B	+73.4%	+42.1%	+267.4%	+136.0%	−11.8%	−7.9%	−40.0%	−25.1%
Qwen3-4B	+19.2%	+14.6%	+184.1%	+103.4%	−5.2%	−3.6%	−46.3%	−35.9%

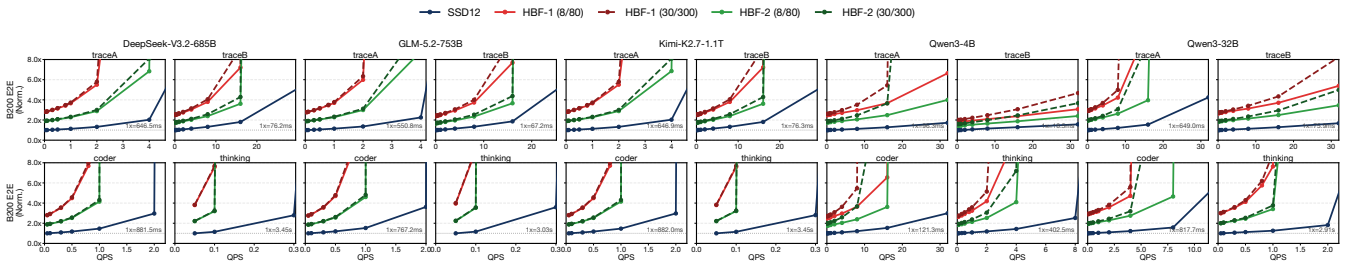


Figure 5: End-to-end latency versus load on B200, normalized within each panel to the low-load SSD12 value (1×), across five models and four traces.

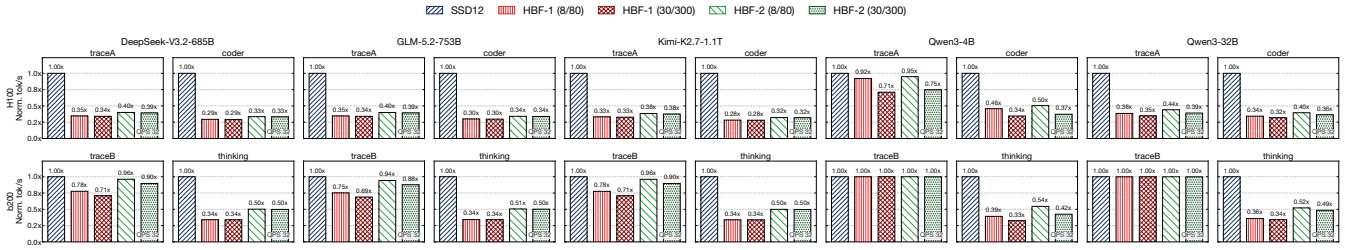


Figure 6: Delivered token throughput normalized to SSD12 (1×) for H100 (top) and B200 (bottom), across five models and four traces.

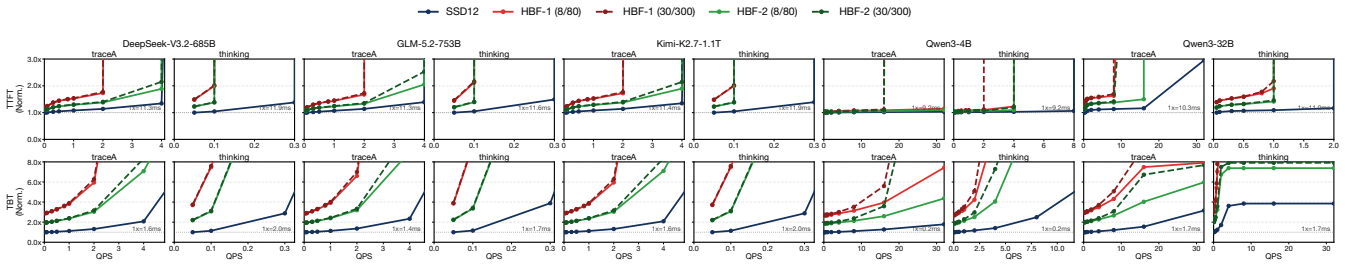


Figure 7: Time-to-first-token (top) and time-between-tokens (bottom) versus load, normalized within each panel to the low-load SSD12 value (1×), for representative traces on five models.

Moving compute into the device raises neither term of f. It enlarges neither the resident fraction nor the per-step work, so C1 survives the intervention. The conclusion is not that base-die compute is useless but that it is the wrong lever for this bottleneck. It accelerates a small, cold, microsecond-latency slice, so base-die silicon is better spent on the control plane than on a general attention engine. The next condition turns to what a swap costs on the write side.

Table 6: Finding 2 NMP configuration.

Parameter	Value
Model / cluster	DeepSeek-V3.2-685B, 8×H100 (TP8, DP1)
Memory preset	HBF2, 12 TB shared HBF tier
HBF latency	12 μs read, 120 μs write
Modes	HBF base-die NMP, HBM base-die NMP
NMP type	24 FP16 MAC arrays at 600 MHz
Internal-stack	
NMP bandwidth	896 GB/s
QPS tiers	0.05, 0.1, 0.2, 0.5, 1, 2, 4, 8, 16, 32

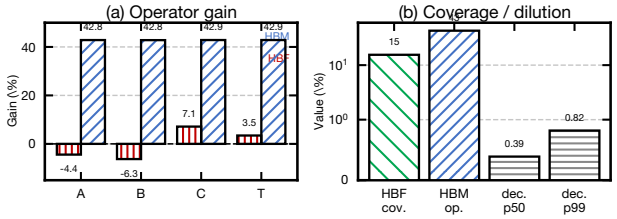


Figure 8: Base-die NMP operator leverage, HBF coverage, and end-to-end dilution. A–T denote traceA, traceB, coder, and thinking.

Table 7: Per-trace NMP leverage and coverage.

Trace	HBF-NMP mean	HBF-NMP QPS≤1	HBM-NMP mean	HBF residency
`traceA`	−4.43%	−21.67%	42.84%	26.62%
`traceB`	−6.27%	−14.11%	42.81%	13.63%
`coder`	7.13%	−1.68%	42.85%	17.64%
`thinking`	3.45%	1.96%	42.85%	4.02%

Finding 2

Base-die attention compute cannot overcome HBF's limited KV coverage, so its silicon is better spent on control rather than general attention.

7 C2: Reads Outweigh Writes

Condition C2 requires HBF to be a read-heavy tier, $\rho > \rho^\star \gtrsim 1$ (Eq. (8)). Finding 3 measures the reads-per-write the hierarchy delivers, and Finding 4 asks whether the standard SSD write optimization can lower the write cost C.

7.1 Finding 3: SSD-Style Hierarchy Makes HBF Write-Heavy

Finding 1 showed the flash service sits off the critical path. Whether it earns its place is a separate question. Every byte HBF serves must first be written there, so C2 asks whether those writes are repaid by later reads. A hierarchy tuned for reuse could land on either side of that threshold.

We replay each of the four production traces for two hours and measure the write and read byte counts at the flash tier directly. Their ratio converts to the delivered ρ that C2 tests. We also record the prefix hit rate, which separates cache effectiveness from flash-tier traffic.

Table 8 reports all three quantities. Every trace writes more to the flash tier than it reads back, from 1.14× (`coder`) to 4.90× (`traceB`), so ρ ranges from 0.20 to 0.88, below break-even on every trace and several-fold below on the write-heaviest. The prefix hit rate is 38–53%, so reuse is found even where the flash tier reads little back.

Table 8: Flash-tier traffic over a full two-hour replay.

Metric	`traceA`	`traceB`	`coder`	`thinking`
Write/read ratio	2.14×	4.90×	1.14×	2.20×
ρ (reads/write)	0.47	0.20	0.88	0.45
Prefix hit rate	38.21%	52.46%	45.58%	44.12%

Reuse distribution across blocks. The two observations reconcile through the reuse distribution, not the average. Reuse is extremely skewed. The top decile of unique blocks supplies 64% (`traceA`) to 95–100% (`thinking`, `traceB`) of all reuse reads, with Gini 0.80–1.0. Those hot blocks are the ones the near tier keeps resident, so those hits are served from HBM or GDDR, and what reaches HBF is the long cold tail, written once and rarely read again. `traceB` is the extreme, with only 6% of blocks ever reused, so almost everything it sends HBF is write-once and its ratio is highest. A cache tier that mostly writes is therefore the hierarchy *doing its job*, not a misconfiguration. It keeps reuse where it is fastest to serve and hands the tier that should hold read-mostly data a write-mostly stream.

Under save/restore pooling, ρ depends on both object class and placement. Stable weights and static tensors support many reads per write ($\rho \gg 1$), and shared-prefix KV can amortize one write over many requests. Private decode KV grows token by token and dies when the request finishes, leaving few opportunities for a later restore. The traces quantify cross-request reuse: The fraction of blocks ever reused across requests is 46.8%, 6.0%, 39.0%, and 14.3%. The pooling policy determines how much of that reuse reaches the flash tier. This is why H3 and HAVEN place stable, read-mostly objects in HBF [7, 9]. The problem is not that HBF cannot hold KV, but that the SSD-style hierarchy routes the least reusable KV to the device least able to absorb writes.

Finding 3

The two-tier hierarchy keeps reusable KV in the near tier and sends HBF a write-heavy cold stream whose writes are rarely repaid.

7.2 Finding 4: SSD Write-Batching helps little for HBF

Finding 3 establishes ρ under the unchanged pooling policy, so the remaining lever in Eq. (8) is the write cost C. SSD KV serving already has the standard tool for lowering it. Mooncake-style batching coalesces many small KV writes into large buckets, amortizing the drive's slow per-operation I/O and turning scattered writes into near-sequential transfers. Whether that win is a property of the optimization or of the medium it was built for decides whether it carries onto HBF.

We run the identical bucketing mechanism on both media over a shared `traceA`/`traceB` matrix, holding HBF's random and sequential bandwidth *equal* so batching can help only by amortizing fixed write-operation latency. We compare three arms, no-batch, the Mooncake bucketed path, and a single-key control that pays the bucketing cost without grouping anything.

Table 9 and Fig. 9(a) give the paired result. On SSD-12, request p50 falls by 8.9–21.1% at every block size. The same mechanism on HBF-2 recovers 2.69% overall, 5.2× smaller, and the benefit is concentrated at the smallest block, +9.2% at 16 tokens, fading to +0.72% at 128 and turning *negative* (−1.85%) at 512. The SSD gain is flat across block size; the HBF gain decays to zero and then costs.

Composition of the write cost. The contrast follows from what each medium's write cost is made of. On SSD a write is dominated by high fixed per-operation overhead and a random-access penalty, and usable bandwidth is low relative to demand. Batching attacks all three at once, so its win is large and survives coarse blocks. Our HBF service model isolates fixed-overhead amortization: random and sequential bandwidth are equal, so bucketing gains no scattered-to-sequential bandwidth advantage. The only lever left is amortizing fixed operation latency, which pays off only when a bucket is dense with tiny blocks, about 94 blocks per bucket at 16 tokens but 4 at 512 (Fig. 9(b)). The single-key control makes that residue explicit. With the batching path enabled but no grouping it is *worse* than no-batch on every metric, because it pays bucketing overhead for no amortization.

SSD-style buckets amortize fixed I/O overhead. OCP specifies complete 4 KiB page writes and sequential programming. For maximum parallelism, banks within a die use the same page number [1, §§5.4, 11.2.2]. Larger buckets preserve the write-heavy stream. Their measured benefit is too small to rescue C2.

Finding 4

SSD-style bucketing delivers little benefit under the evaluated HBF service model and leaves the write-heavy pooling stream unchanged.

8 C3: Read/Write Performance Sustains

Condition C3 requires the bandwidth the workload receives to stay near the datasheet peak. Finding 5 measures what an HBF stack sustains at run time, and Finding 6 measures what the write side costs over the device's life.

Table 9: Paired batching benefit (batched vs. matched no-batch), SSD-12 vs. HBF-2 on the shared `traceA`/`traceB` matrix.

	Req. p50 gain		Token/s gain	
Block (tokens)	SSD-12	HBF-2	SSD-12	HBF-2
overall	+13.98%	+2.69%	+2.04%	+0.31%
16	+21.14%	+9.20%	+3.08%	+0.91%
128	+11.88%	+0.72%	+1.56%	+0.06%
512	+8.92%	−1.85%	+1.48%	−0.04%
HBF-2 controls (no SSD analogue)				
batched, all blocks	—	+1.76%	—	+0.31%
1-block control	—	−4.25%	—	−0.37%

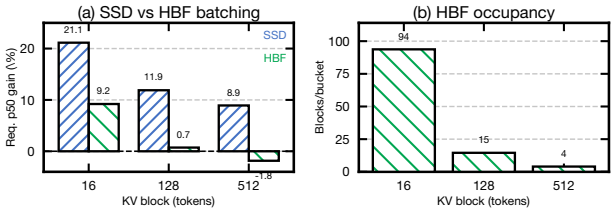


Figure 9: Batching gain does not transfer from SSD to HBF. (a) Request-p50 reduction from Mooncake batching on SSD-12 vs. HBF-2 across KV block sizes; SSD gains hold everywhere while HBF fades to negative. (b) HBF-2 bucket occupancy collapses as blocks grow, removing the only latency HBF batching can amortize.

8.1 Finding 5: Write-Heavy Traffic Overheats HBF and Throttles Bandwidth

Finding 3 established that the flash stream a KV hierarchy generates is write-heavy, and writes cost more power than reads. What sustained bandwidth an HBF stack can hold under that traffic is therefore open; the interface peak bounds it but does not answer it.

Thermal model and throttling policy. A 3D-ICE [23] model of a 16-Hi TLC stack, driven by the access energy of a 16-token KV block, sweeps sustained single-stack bandwidth (§4). The thermal policy in Section 4 closes the hottest, highest-traffic planes and redirects writes to cooler ones.

Dynamic power scales linearly with bandwidth, and the stack hits the 80°C limit at only 202.27 GB/s and 53.72 W, far below interface peak, so it cannot run continuously above that point (Fig. 10(a)). In the replay, the unconstrained run holds peak bandwidth through a burst, while the thermal controller drops the constrained run to the 202 GB/s safe point and holds temperature near 80°C (Fig. 10(b)–(c)).

Why the stack runs hot. Delivered bandwidth is not a constant. It varies in *time* as hot regions throttle and cool, and in *space* as the runtime steers traffic away from the hottest planes. The cause traces back to Finding 3. Because reuse stays in the near tier and writes dominate the flash stream at higher power per access, the

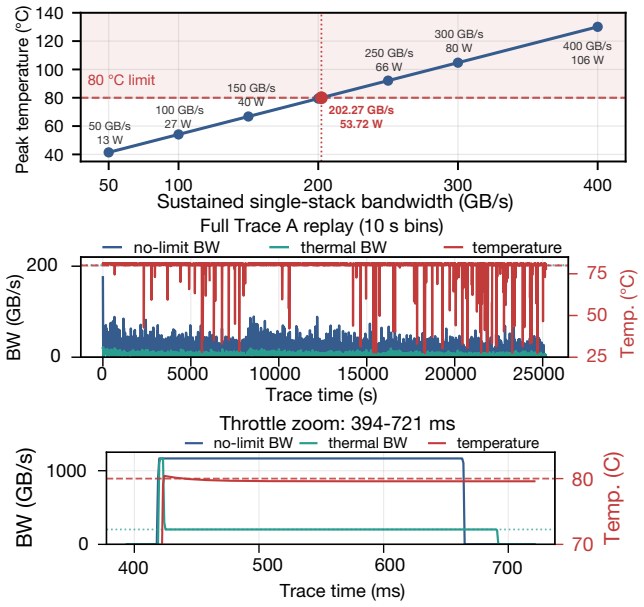


Figure 10: (a) Dynamic power and peak temperature vs. single-stack bandwidth. (b) Full replay: thermal control caps delivered bandwidth. (c) Throttle zoom: temperature stays near 80°C while bandwidth is capped.

device is pushed to its limit by exactly the write-heavy traffic the hierarchy sends it. A read-mostly workload at the same bandwidth would run cooler.

A scheduler that assumes a fixed flash bandwidth will mispredict service time and build queues it did not expect. The OCP specification likewise treats thermal state as part of the host contract: severe throttling halts data I/O while retaining maintenance service [1, §9.2]. The unchanged pooling runtime therefore needs more than a peak-bandwidth number to sustain its write-heavy stream.

Finding 5

Write-heavy KV traffic drives HBF into thermal throttling, making delivered bandwidth lower and less predictable than its rated peak.

8.2 Finding 6: HBF Wears Out Sooner Under Write-Heavy KV Pooling

Finding 5 measured what the write-heavy stream costs in the moment, when its power throttles delivered bandwidth. The same stream also has a cumulative cost. Every write spends part of a finite program/erase budget. We compare the trace-measured write volume against the HBF and capacity-matched SSD endurance budgets defined in Section 4.

Every trace writes past *both* envelopes (48–140 TB/day), so under linear TBW exhaustion the nominal HBF tier lasts a constant 0.56× the SSD pool's life across all four traces (Fig. 11). Coarsening KV blocks from 16 to 128 to 512 tokens raises the apparent prefix hit rate yet *increases* HBF write volume from 188 to 237 to 258 TB (1.38×).

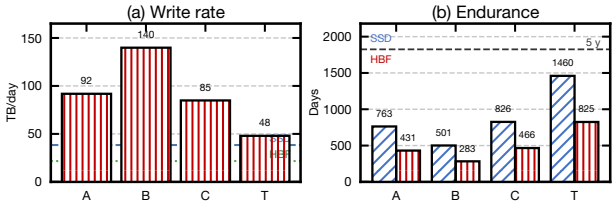


Figure 11: KV write volume and endurance budget. A–T denote traceA, traceB, coder, and thinking.

Why the shortfall is structural. HBF is attached like memory but behaves like flash. OCP assigns ECC, bad-block handling, and write accumulation to the base die. Wear leveling can run in the base die or on the host [1, §§4.4, 5.4, 11.4]. Wear leveling distributes wear but leaves the pooling stream's programming demand intact. Unlike an SSD's internal reclamation machinery [4], OCP delegates live-data movement and reclamation to the host. Zone remapping requires invalid data and quiesced reads before rewriting [1, §11.4]. OCP specifies host refresh for data age and read-disturb counts, and 24-hour powered-on retention at 85°C [1, §§9, 11.5]. These responsibilities make write volume, wear distribution, and data lifetime explicit inputs to placement, rather than properties hidden by a peak-bandwidth number. Block granularity does not route around that state. Small blocks maximize the number of write operations. Larger blocks increase bytes written and write amplification: partial reuse can still require rewriting the entire block. Both keep the device on the wrong side of Eq. (8).

Our trace-driven endurance comparison shows that HBF wears out sooner than the capacity-matched SSD pool it replaces, exposing the endurance cost C_{life} in Eq. (6). Writing short-lived KV into HBF pays twice: programming must complete, and programs consume the endurance budget.

Finding 6

Under unchanged SSD-style KV pooling, HBF wears out sooner than the capacity-matched SSD pool: the measured write-heavy stream exhausts its endurance budget faster.

9 Discussion and Related Work

9.1 Synthesis

The six findings falsify the three conditions of Section 5 in turn, and together they mark where HBF does help. An object belongs in HBF only when its restore delay is the serving bottleneck (C1), its reads outweigh its writes ($\rho > \rho^{\star}$, C2), and the bandwidth it needs is sustainable under the package's power and endurance budget (C3). Fig. 12 places representative object classes on the two axes that decide this (exposed stall and reads-per-write), with sustainable bandwidth shifting the boundary. Transient decode KV sits in the low-exposure, low-ρ corner where all three conditions fail; weights and shared prefixes sit in the opposite corner, which is where H3 and HAVEN place their HBF-resident state.

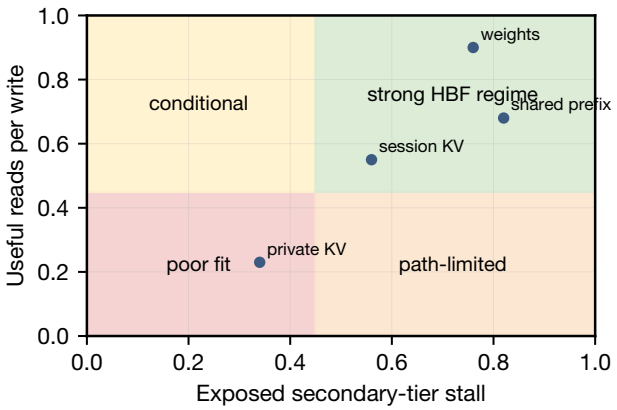


Figure 12: Placement map implied by the findings: exposed stall (C1) against reads-per-write (C2), with sustainable bandwidth (C3) shifting the boundary.

The three conditions use device-agnostic quantities: exposed critical-path fraction, reads per write, and sustained bandwidth. The same map therefore applies to other package-local far tiers such as CXL-attached or near-data flash, not only to HBF. The map gives the axes and the break-even rule of Eq. (8) rather than a calibrated boundary across the latency×bandwidth plane. A working runtime requires critical-path-aware access, reuse-aware admission, and thermal-aware write budgeting. The OCP specification reinforces this object-aware direction by describing separate weight and KV channels to manage endurance and capacity utilization [1, §13.3.3]. Our measurements establish why an unchanged SSD-style pool is a poor match.

9.2 Related Work

KV-cache management. PagedAttention enables block allocation and prefix sharing [15], quantization and compaction shrink KV size [8, 18, 28], and Prompt Cache and CachedAttention reuse state across prompts [5, 6]. Mooncake, LMCache, Strata, and Tutti offload KV across CPU, SSD, and network tiers with GPU-assisted I/O [17, 20, 21, 27], and DeepSeek's disk context cache evidences persisted prefix reuse [3]. We ask how this SSD-offload stack behaves when its backing medium moves into the accelerator package.

HBF. Prior studies explore HBF as either a read-mostly extension or the main GPU memory. H3 places read-only weights and shared precomputed KV in HBF while retaining generated state in HBM [7]; HAVEN stores a vector collection in HBF and reranks near the device [9]. FlashAccel adds six HBF stacks to an HBM-based GPU and reports throughput 2.54× that of its HBM-only baseline [25]. The POSTECH study evaluates both one HBM plus seven HBF stacks and an all-HBF configuration with eight HBF stacks. Weights and KV reside in flash; intermediate data use the retained HBM or base-die SRAM, respectively [22]. These configurations explore HBF as primary memory, distinct from the HBF roadmap's GDDR–HBF and shared HBM–HBF organizations evaluated here. Our focus is the HBF-1/HBF-2 package tradeoff under an SSD-style KV hierarchy and production-trace replay. The studies thus address different organizations, data paths, and serving conditions.

Flash reliability. SSD controllers manage mapping, reclamation, and wear [4]. The OCP specification divides management between the base die and the host, which handles data movement and refresh [1, §§11.4–11.5]. NAND retention depends on wear and temperature [2].

9.3 Limitations

The hardware profiles follow the HBF roadmap's near-term organizations and the service parameters in Section 4. The architecture comparison measures the joint effect of the near tier and backing tier under the same pooling policy; the latency sweep isolates media-service sensitivity. Thermal and endurance results use the stated temperature policy and TLC write budget, while cost is expressed through package-resource trade-offs. We take the host/device contract directly from OCP to explain the layout, write-management, and sustained-service requirements of this deployment.

10 Conclusion

For the HBF roadmap organizations studied here—HBF-1 (2028) and HBF-2 (2030)— replacing an SSD KV tier with HBF does not improve LLM serving; under the same Mooncake-style runtime it makes serving worse. Our model explains why: a faster far tier helps only when read I/O is the serving bottleneck (C1), reads outweigh writes (C2), and its bandwidth is sustainable (C3), and all three fail for transient KV behind an SSD-style connector. HBF sucks as a drop-in SSD KV pool under this unmodified runtime: serving slows down, and the trace-driven endurance comparison shows that HBF wears out sooner than the capacity-matched SSD pool it replaces. The OCP specification defines page-write and layout rules and lifecycle controls. These reinforce the need to change the pooling policy and data path, not merely replace the backing medium.

References

[1] Chinnakrishnan Ballapuram and Dongsop Lee. 2026. *High Bandwidth Flash (HBF) High-Level Base Die Specification.* Open Compute Project Specification Version 0.7.0. Sandisk Corporation and SK hynix. August 3, 2026.

[2] Yu Cai, Yixin Luo, Erich F. Haratsch, Ken Mai, and Onur Mutlu. 2015. Data Retention in MLC NAND Flash Memory: Characterization, Optimization, and Recovery. In *Proceedings of the 21st IEEE International Symposium on High Performance Computer Architecture*. 551–563. doi:10.1109/HPCA.2015.7056062

[3] DeepSeek. 2026. Context Caching on Disk. DeepSeek API Documentation. Accessed: 2026-07-15. https://api-docs.deepseek.com/guides/kv_cache/

[4] Eran Gal and Sivan Toledo. 2005. Algorithms and Data Structures for Flash Memories. *Comput. Surveys* 37, 2 (2005), 138–163. doi:10.1145/1089733.1089735

[5] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 111–126. https://www.usenix.org/conference/atc24/presentation/gao-bin-cost

[6] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 325–338. https://proceedings.mlsys.org/paper_files/paper/2024/file/a66caa1703fe34705a4368c3014c1966-Paper-Conference.pdf

[7] Minho Ha, Euiseok Kim, and Hoshik Kim. 2026. H3: Hybrid Architecture Using High Bandwidth Memory and High Bandwidth Flash for Cost-Efficient LLM

Inference. *IEEE Computer Architecture Letters* 25, 1 (2026), 49–52. doi:10.1109/LCA.2026.3660969

[8] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[9] Po-Kai Hsu, Weihong Xu, Qunyou Liu, Tajana Rosing, and Shimeng Yu. 2026. HAVEN: High-Bandwidth Flash Augmented Vector Engine for Large-Scale Approximate Nearest-Neighbor Search Acceleration. arXiv:2603.01175 [cs.AR] https://arxiv.org/abs/2603.01175

[10] Yanpeng Hu, Yiwei Yang, Yuanwu Zhu, Kexin Chu, Yusheng Zheng, Andi Quinn, and Wei Zhang. 2026. HBFSim: Fast and Faithful Simulation of High-Bandwidth Flash Under Real GPU Execution. arXiv:2609.09800v1 [cs.AR] https://arxiv.org/abs/2609.09800v1

[11] Joungho Kim. 2026. HBF Technology, Workload Analysis and Roadmap. Presentation, TeraByte Interconnection and Package Laboratory, KAIST, February 10, 2026.

[12] Seeyeon Kim, Juhyeong Jin, and Joo-Young Kim. 2026. Beyond Capacity: Scalable MoE LLM Inference via High-Bandwidth Flash with Direct GPU and HBM Paths. arXiv:2608.14333v1 [cs.AR] https://arxiv.org/abs/2608.14333v1

[13] KIOXIA America, Inc. 2024. KIOXIA SSDs for Supermicro Servers: Data Sheet. Version 1.0. https://americas.kioxia.com/content/dam/kioxia/en-us/business/ssd/oem/supermicro/asset/KIOXIA_Supermicro_SSD_Data_Sheet_Global.pdf

[14] KVCache.AI Team. 2026. Mooncake SSD Offloading: Breaking Through Memory Limits to Expand KV Cache Capacity. Mooncake Design Documentation. Accessed: 2026-07-19. https://kvcache-ai.github.io/Mooncake/design/ssd-offload.html

[15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[16] Linux Kernel Documentation. 2026. io_uring Subsystem Documentation. Accessed: 2026-07-15. https://docs.kernel.org/io_uring/

[17] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. 2025. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665 [cs.DC] https://arxiv.org/abs/2510.09665

[18] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

[19] Geraldo F. Oliveira, Arash Tavakkol, Xiangyu Zhu, Ahmet Caner Yüzügüler, Vamanan Arulchelvan, Lukas Cavigelli, Renzo Andri, Mohammad Sadrosadati, Jia Xinglei, Onur Mutlu, Zhou Ke, Shai Bergman, and Ji Zhang. 2026. FLINT: Efficiently Leveraging High Bandwidth Flash for Capacity-Scalable LLM Inference Acceleration. arXiv:2608.25062v1 [cs.AR] https://arxiv.org/abs/2608.25062v1

[20] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. MOONCAKE: trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies* (Santa Clara, CA, USA) *(FAST '25)*. USENIX Association, USA, Article 10, 16 pages.

[21] Shi Qiu, Yifan Hu, Xintao Wang, Wenhao Zhu, Jianqin Yan, Hao Chen, Kaiqiang Xu, Kai Chen, and Yiming Zhang. 2026. Tutti: Making SSD-Backed KV Cache Practical for Long-Context LLM Serving. arXiv:2605.03375 [cs.DC] https://arxiv.org/abs/2605.03375

[22] Dowon Son, Yonggon Park, Hyunuk Cho, Hyungkyu Ham, Onur Mutlu, Sungjin Lee, Gwangsun Kim, and Jisung Park. 2026. Exploring High-Bandwidth Flash for Modern LLM Inference: Opportunities and Challenges. *IEEE Computer Architecture Letters* (2026). doi:10.1109/LCA.2026.3705817

[23] Arvind Sridhar, Alessandro Vincenzi, Martino Ruggiero, Thomas Brunschwiler, and David Atienza. 2010. 3D-ICE: Fast Compact Transient Thermal Modeling for 3D ICs with Inter-tier Liquid Cooling. *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* (2010), 463–470. doi:10.1109/ICCAD.2010.5653749

[24] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. In *Proceedings of the 2025 USENIX Annual Technical Conference*. USENIX Association, 465–480. https://www.usenix.org/conference/atc25/presentation/wang-jiahao

[25] Xinyu Wang, Yalong Xue, Xiaotian Sun, Xiaoyu Zhang, Chunmeng Dou, Xueqi Li, and Xiaoming Chen. 2026. FlashAccel: Leveraging High-Bandwidth Flash for High-Throughput LLM Inference. arXiv:2607.10186 [cs.AR] https://arxiv.org/abs/2607.10186

[26] Feiyang Wu, Zhuohang Bian, Guoyang Duan, Tianle Xu, Junchi Wu, Teng Ma, Yongqiang Yao, Ruihao Gong, and Youwei Zhuo. 2025. TokenSim: Enabling Hardware and Software Exploration for Large Language Model Inference Systems. arXiv:2503.08415 [cs.DC] https://arxiv.org/abs/2503.08415

[27] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. 2025. Strata: Hierarchical Context Caching for Long Context Language Model Serving. arXiv:2508.18572 [cs.DC] https://arxiv.org/abs/2508.18572

[28] Yanqi Zhang, Yuwei Hu, Runyuan Zhao, John C.S. Lui, and Haibo Chen. 2025. DiffKV: Differentiated Memory Management for Large Language Models with Parallel KV Compaction. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association.